Storage Management

Week 13

Operating Systems Ch 11 – Mass-Storage Structure

Today's Schedule

Hour	Content
1st	**Quiz (Beginning)** → Theory Lecture (Part 1)
2nd	Theory Lecture (Part 2)
3rd	Hands-on Lab

Part 1

HDD Structure & Access Time

Overview of Mass-Storage Structure

Classification of mass-storage devices in modern computers

Category	Examples	Characteristics
Secondary Storage	HDD, SSD	Primary nonvolatile storage
Tertiary Storage	Magnetic Tape, Optical Disk	Slow but high capacity, used for backup
NVS (Nonvolatile Storage)	All nonvolatile devices	Data persists even when power is off

Chapter Objectives:

- Understand the physical structure and characteristics of storage devices
- Analyze I/O scheduling algorithms
- Understand OS storage device management services (including RAID)

HDD Structure

Physical structure of a Hard Disk Drive

```
        Spindle (rotation axis)
            |
    ┌──────────────┐
    │  Platter     │   ←— Disk platter (both sides used)
    │  ┌─────────┐ │
    │  │ Track   │ │   ←— Concentric circular tracks
    │  │ Sector  │ │   ←— Smallest unit of a track (512B~4KB)
    │  └─────────┘ │
    └──────────────┘
            ↕
    Disk Arm + Read-Write Head
```

Component	Description
Platter	Disk platter where data is stored (both sides usable)
Track	Concentric circles on the platter surface

HDD Physical Details

Read-Write Head and Disk Arm

- The read-write head flies over each platter surface on a very thin air/helium cushion
- All heads are attached to a single disk arm and move simultaneously
- Platter surfaces are coated with magnetic material; data is recorded/read magnetically

```
Platter 1  ——— Head 1 (surface 1)
            ——— Head 2 (surface 2)
Platter 2  ——— Head 3 (surface 3)
            ——— Head 4 (surface 4)
Platter 3  ——— Head 5 (surface 5)
            ——— Head 6 (surface 6)
     ↑
   Spindle        ←— Disk Arm (moves all heads together)
```

Head Crash

- An accident where the head contacts the platter surface
- Damages the magnetic material on the platter surface → data is unrecoverable
- The entire disk must be replaced (RAID can protect data)

HDD Rotation Speed and Capacity

Spindle Rotation Speed (RPM)

RPM	Use Case	Time per Revolution
5,400	Laptops, low power	11.1 ms
7,200	Desktops, general servers	8.3 ms
10,000	High-performance servers	6.0 ms
15,000	Enterprise SAS	4.0 ms

- Transfer rate: data transfer speed between drive and computer
- Actual transfer speed is always lower than nominal speed
- Drive controllers include built-in DRAM buffers for performance improvement
- Common platter diameters: 1.8 ~ 3.5 inches

HDD Access Time

Access Time = Seek Time + Rotational Latency + Transfer Time

Component	Description	Magnitude
Seek Time	Time for the arm to move to the desired cylinder	Largest (several ms)
Rotational Latency	Waiting for the desired sector to rotate under the head	Medium
Transfer Time	Time for data to actually be read or written	Smallest

```
Seek Time         Rotational Latency    Transfer Time
──────────→       ────────────────→      ─────→
arm movement       sector rotation wait   data transfer
```

Seek Time dominates the total access time

- Goal of HDD scheduling: **minimize seek time**
- Reorder requests to reduce arm movement distance

HDD Access Time Calculation Example

Access time calculation for a 7,200 RPM disk

Rotational Latency Calculation:

- 7,200 RPM → 1 revolution = 60/7,200 = 8.33 ms
- Average rotational latency = 8.33 / 2 = **4.17 ms** (average half-revolution wait)

Typical access time breakdown:

```
Average Seek Time:          8 ms
Average Rotational Latency: 4.17 ms
Transfer Time (4KB):        ~0.01 ms
──────────────────────────────────
Total Access Time:          ~12.18 ms
```

- Seek time accounts for approximately 66% of the total
- Only about 82 random I/Os per second possible (= 1000/12.18)
- Sequential I/O is much faster since seek/rotation are nearly eliminated
- HDD has a very large performance gap between random I/O and sequential I/O

Part 2

NVM Devices (SSD / NAND Flash)

NVM Devices – Overview

SSD (Solid State Drive): Based on NAND Flash memory

Property	HDD	SSD
Mechanical parts	Yes (arm, spindle)	None
Seek Time	Yes	None
Rotational Latency	Yes	None
Random vs Sequential	Large difference	Small difference
Speed	Slow	Fast
Durability	Mechanical wear	Limited write cycles

NAND Flash Characteristics

Key characteristics of NAND Flash semiconductors

Read/Write/Erase units differ:

Operation	Unit	Speed
Read	Page (typically 4KB~16KB)	Fastest
Write	Page	Slower than read
Erase	Block (group of multiple pages)	Slowest

Overwrite is not possible:

- Cannot directly overwrite an already-written page
- Must erase at the block level first → then write is possible

NAND Flash Controller Algorithms

Flash Translation Layer (FTL)

- Manages logical block address → physical page mapping
- Tracks valid/invalid page states
- The OS simply reads and writes logical blocks; the controller manages internals

Garbage Collection:

- When the entire SSD is nearly full, cleans up blocks containing invalid data
- Copies valid data to another location → erases the invalid block → reuses it
- This process affects performance

```
Block A: [Valid][Invalid][Invalid][Valid] → copy only valid data
Block B: [Valid][Valid][Valid][Free]       ← move here
Block A: [Erased] → available for reuse
```

Over-provisioning:

- Reserves approximately 20% of total capacity as spare space

Wear Leveling & TRIM

Wear Leveling

- NAND Flash has a limited number of erase cycles per cell
- If erases concentrate on specific blocks, those blocks wear out first
- The controller **evenly distributes** erases across all blocks

```
Without Wear Leveling:     With Wear Leveling:
Block 0: ████████ (heavy use)    Block 0: ████ (even)
Block 1: ▌ (barely used)        Block 1: ████ (even)
Block 2: ██████ (heavy use)     Block 2: ████ (even)
Block 3: ▌ (barely used)        Block 3: ████ (even)
```

TRIM:

- When a file is deleted, the OS notifies the SSD controller that the block is no longer in use
- The controller pre-erases that block to improve subsequent write performance
- Without TRIM, the controller cannot know which data is invalid

Write Amplification:

NVM Scheduling

SSDs have no seek, so HDD scheduling is unnecessary

Property	HDD	SSD
Random Read	Slow (seek required)	Fast (~100K IOPS)
Sequential Read	Relatively fast	Fast
Random Write	Slow	Fast but variable
Sequential Write	Relatively fast	Fast

- SSDs use a simple **FCFS** policy (Linux NOOP scheduler)
- Adjacent requests are **merged** to optimize transfer efficiency
- Read service time is uniform, but write service time is **non-uniform**
  - Varies depending on garbage collection, device utilization, and wear level

Volatile Memory as Storage

RAM Drive (RAM Disk)

- Uses a portion of DRAM as a storage device
- A device driver allocates a DRAM region and presents it as a block device
- Fastest file creation/read/write/delete speed

OS	RAM Drive
Linux	`/dev/ram`, tmpfs (`/tmp`)
macOS	Created via `diskutil` command
Windows	Third-party tools

- **Volatile**: data is lost on system restart
- Use cases: high-speed temporary file storage, data sharing between programs
- Linux initrd: used as a temporary root file system during early boot

Part 3

Secondary Storage Connection & Address Mapping

Secondary Storage Connection Methods

Interfaces for connecting storage devices to the system

Interface	Characteristics
SATA	General purpose, low cost, up to ~600MB/s, most common
NVMe (PCIe)	High speed, low latency, optimized for SSDs, direct PCIe connection
SAS	Server-grade, high reliability, cheaper than FC
USB	External storage, universal connectivity
Fibre Channel (FC)	SAN environments, high-speed networked storage

Host Controller (HBA): bus controller on the computer side **Device Controller:** built-in controller in each storage device

Address Mapping – LBA

Logical Block Address (LBA)

- Abstracts the storage device as a **1-dimensional array of logical blocks**
- Each logical block maps to a physical sector or NVM page

```
Application        OS              Storage Device
──────────→    ──────────→       ──────────────────
File name       LBA number         Physical location
               (0, 1, 2, ... )    HDD: Cylinder-Head-Sector (CHS)
                                  NVM: Chip-Block-Page
```

LBA Mapping in HDD:

- Sector 0 → first sector of the first track of the outermost cylinder
- Mapped sequentially across tracks → cylinders → inward
- In practice, it is more complex due to bad sector replacement, variable sector counts, etc.

CAV vs CLV:

Part 4

HDD Scheduling Algorithms

HDD Scheduling – Overview

Purpose of disk scheduling

- **Minimize access time** (especially seek time)
- **Maximize data transfer bandwidth**
- Reorder the request queue to reduce arm movement distance

Information included in an I/O request:

1. Whether it is Input or Output
2. Open file handle (file identifier)
3. Memory address (target memory address for transfer)
4. Amount of data (size of data to transfer)

When the disk is busy, new requests are stored in a **pending queue**

- Queue order can be optimized for performance improvement
- Modern drives use LBA, so closer LBAs are assumed to be physically closer

HDD Scheduling – FCFS

Process requests in **arrival order** (First-Come, First-Served)

Request queue: **98, 183, 37, 122, 14, 124, 65, 67** (head start: 53)

```
head
 53 → 98 → 183 → 37 → 122 → 14 → 124 → 65 → 67
```

Total head movement distance: = |53-98| + |98-183| + |183-37| + |37-122| + |122-14| + |14-124| + |124-65| + |65-67| = 45 + 85 + 146 + 85 + 108 + 110 + 59 + 2 = **640 cylinders**

Advantages	Disadvantages
Simple to implement	Head traverses entire disk back and forth
Fair (first-come, first-served)	Total seek distance is very large
No starvation	Poor performance

FCFS – Movement Path Visualization

```
Cylinder:  0    14   37   53   65 67   98   122 124   183   199
           |    |    |    |    |  |    |    |   |     |     |
Step 0     |    |    |    ●    |  |    |    |   |     |     |  (start: 53)
Step 1     |    |    |    └────────────●    |   |     |     |  → 98 (+45)
Step 2     |    |    |         |  |    └──────────────●     |  → 183 (+85)
Step 3     |    |    ●──────────────────────────────┘     |  → 37 (+146)
Step 4     |    |    └──────────────●   |     |     |     |  → 122 (+85)
Step 5     |    ●──────────────────┘   |     |     |     |  → 14 (+108)
Step 6     |    └──────────────────────●     |     |     |  → 124 (+110)
Step 7     |    |    |    |    ●──────────┘     |     |     |  → 65 (+59)
Step 8     |    |    |    |    | ●   |    |    |     |     |  → 67 (+2)
```

122 → 14 → 124: the arm exhibits extreme back-and-forth **wild swing** behavior

- If 37 and 14 were processed first, then 122 and 124, it would be much more efficient

HDD Scheduling – SCAN (Elevator)

Moves in one direction to the end while servicing requests, then **reverses direction**

Request queue: **98, 183, 37, 122, 14, 124, 65, 67** (head start: 53, initial direction: decreasing)

```
head
 53 → 37 → 14 → [0] → 65 → 67 → 98 → 122 → 124 → 183
                  ↑
             direction reversal (reverse after reaching the end)
```

Total head movement distance: = (53-37) + (37-14) + (14-0) + (0-65) + (65-67) + … + (124-183) = 16 + 23 + 14 + 65 + 2 + 31 + 24 + 2 + 59 = **236 cylinders** (significantly reduced compared to FCFS's 640)

Works on the same principle as an elevator algorithm

- Services all requests along the path while moving in one direction
- When reaching the end, reverses and services the remaining requests

SCAN – Characteristics and Issues

Advantages of SCAN

- Much more **efficient** than FCFS (reduced arm movement distance)
- Good performance in most cases

Disadvantages of SCAN – Uneven waiting times

- Requests at the middle position are **favored** over those at the ends (two chances to be passed)
- Right after the head reaches one end: many recently serviced requests near that end
- Requests near the opposite end wait the longest

```
← head movement direction
[0]──────────────────────────────[199]
     ↑                               ↑
longest waiting requests    recently serviced
```

To solve this problem → **C-SCAN** was introduced

HDD Scheduling – C-SCAN (Circular SCAN)

Services requests in one direction only; when reaching the end, **immediately moves to the start**

Request queue: **98, 183, 37, 122, 14, 124, 65, 67** (head start: 53, direction: increasing)

```
53 → 65 → 67 → 98 → 122 → 124 → 183 → [199]
                                        ↓ (move to start, no service)
[0] → 14 → 37
```

SCAN vs C-SCAN

Property	SCAN	C-SCAN
Service direction	Bidirectional	Unidirectional
Waiting time	Middle is favored	**Uniform for all positions**
Implementation	Simple	Slightly more complex than SCAN

LOOK and C-LOOK Variants

Practical variants of SCAN/C-SCAN

LOOK: Works like SCAN, but does not go to the end — moves only to the **last request in that direction**

C-LOOK: Works like C-SCAN, but does not go to the end — moves only to the **last request in that direction**

```
SCAN:    53 → 37 → 14 → [0] → 65 → ...     (goes all the way to 0)
LOOK:    53 → 37 → 14 →       65 → ...     (goes only to 14, then reverses)

C-SCAN:  53 → 65 → ... → 183 → [199] → [0] → 14 → 37
C-LOOK:  53 → 65 → ... → 183 →         14 → 37
```

Algorithm	Goes to the end?	Bidirectional service?
SCAN	Yes	Yes
LOOK	No	Yes

Scheduling Algorithm Comparison – Summary

Request queue: 98, 183, 37, 122, 14, 124, 65, 67 / Head: 53 / Disk: 0~199

Algorithm	Total Movement	Characteristics
FCFS	640	Fair but inefficient
SCAN (down)	236	Bidirectional service, efficient
C-SCAN (up)	382	Uniform waiting time
LOOK (down)	208	Improved SCAN, does not go to end
C-LOOK (up)	330	Improved C-SCAN, does not go to end

Algorithm selection criteria:

Linux I/O Schedulers

Major I/O schedulers used in Linux

Deadline Scheduler:

- Separates read and write queues, **prioritizes reads**
- Sorts by LBA order (C-SCAN) for batch processing
- Sets a deadline for each request (default 500ms) → prevents starvation
- Also maintains an FCFS queue to guarantee processing of old requests

NOOP Scheduler:

- Suitable for CPU-bound systems or NVM (SSD) devices
- FCFS + adjacent request merging
- Complex scheduling is unnecessary for SSDs since there is no seek

CFQ (Completely Fair Queueing):

- Default scheduler for SATA HDDs (in some distributions)
- 3 priority queues: Real-time > Best-effort > Idle

Part 5

Error Detection and Correction

Error Detection and Correction

Detecting and correcting data errors that occur during storage/transmission

Technique	Function	Description
Parity Bit	Error detection	Detects single-bit errors, cannot correct
CRC	Integrity verification	Cyclic Redundancy Check, hash-based, detects multi-bit errors
ECC	Error detection + correction	Error-Correcting Code, can correct a small number of bit errors

```
Parity (even parity example):
Data: 10110101  → number of 1s = 5 (odd)
Parity bit = 1    → 10110101 [1]  (total number of 1s becomes even)

ECC:
Data + multiple check bits → locates and corrects error position
Soft error: correctable by ECC → recoverable
Hard error: not correctable by ECC → data loss
```

ECC and Storage Application

ECC usage in HDD and SSD

HDD:

- When writing each sector, ECC is calculated and stored alongside
- On read, ECC is recalculated and compared
- Mismatch → data corruption detected → automatically corrected if only a few bits

SSD (NVM):

- ECC is stored when writing each page
- If frequent correctable errors occur, the page is marked as bad
- That page is no longer used for subsequent writes

```
Write:  Data ──→ [ECC calc] ──→ [Data + ECC] ──→ Storage
Read:   Storage ──→ [Data + ECC] ──→ [ECC recalc]
                                          │
                        Match? ──→ Yes: return normally
                                └→ No:  correctable? ──→ Yes: Soft Error (correct and return)
                                                      └→ No:  Hard Error (I/O error)
```

Part 6

Storage Device Management

Drive Formatting

Three-step process to make a disk usable

```
Low-level Formatting → Partitioning → Logical Formatting
(create physical structure)  (logical division)  (create file system)
```

Step	Description
Low-level Formatting	Creates header, data area, and trailer structure for each sector. Performed at the factory
Partitioning	Divides the disk into one or more partitions (Linux: `fdisk`)
Logical Formatting	Creates a file system on the partition (FAT, ext4, NTFS, etc.)

Low-level format structure:

```
┌──────────┬──────────┬──────────┐
│ Header   │ Data Area │ Trailer  │
```

Partitions and Volumes

Partition:

- A logically divided region of a disk
- Each partition can be used as an independent device
- Linux: `/dev/sda1`, `/dev/sda2`, ...

Volume:

- A single partition, or a logical storage unit combining multiple partitions/devices
- Linux LVM: can combine multiple disks into a single volume
- ZFS: integrates volume management and file system

Volume management process:

```
Physical Disk
```

Part 1 (ext4)	Part 2 (swap)	Part 3 (ext4)

Boot Block

System boot process

1. Power on → **Firmware** (bootstrap loader in NVM flash) executes
2. Firmware reads the **boot block** (MBR/GPT) from the storage device
3. The boot block code loads and executes the **OS kernel**

```
Power On → Firmware (BIOS/UEFI)
                ↓
            Read Boot Block (MBR)
                ↓
            ┌──────────────────────────┐
            │ MBR                      │
            │ ┌──────────┬─────────────┐│
            │ │Boot Code │Partition    │
            │ │          │Table        │
            │ └──────────┴─────────────┘│
            └──────────────────────────┘
                ↓
            Read Boot Sector of Boot Partition
                ↓
            Load and execute OS Kernel
```

- **Boot disk (system disk):** a disk containing the boot partition

Bad Blocks

Bad block management

Causes:

- Manufacturing defects (present since factory shipment)
- Physical damage during use (head crash, etc.)
- Degradation of magnetic material due to aging

Management Method 1 — Sector Sparing (Forwarding):

- The controller replaces bad sectors with **spare sectors**
- Spare sectors are reserved during low-level formatting
- Spares within the same cylinder are used first (minimizing seek)

```
LBA 87 request → Controller checks ECC → Bad sector!
→ Automatically redirected to spare sector
→ Subsequent LBA 87 requests always map to the spare sector
```

Management Method 2 — Sector Slipping:

Part 7

Swap-Space Management

Swap-Space Management

Disk space for extending virtual memory

Purpose:

- When physical memory is insufficient, pages are sent to disk (**swap out**)
- Brought back into memory when needed (**swap in**)
- Modern OSes swap at the **page level**, not the entire process

Swap-Space Size:

- Under-allocation → system aborts processes or crashes
- Over-allocation → just wastes disk space, no major problem
- Old Linux recommendation: 2x physical memory
- Modern systems: less is used since physical memory is sufficient

```
Physical Memory        Swap Space (Disk)

┌─────────────┐        ┌─────────────┐
│ Active Pages │   ↔    │ Swapped Out │
│             │        │ Pages       │
```

Swap-Space Location

Implementation method comparison

Method	Advantages	Disadvantages
Separate raw partition	Fast (no file system overhead)	Fixed size, low flexibility
File-based (swap file)	Easy to resize	File system overhead

Linux:

- Supports **both** swap partitions and swap files
- Multiple swap spaces can be distributed across different disks (I/O bandwidth distribution)
- `mkswap` : create swap area
- `swapon` : activate swap area

```
# Create and activate swap partition
mkswap /dev/sda2
swapon /dev/sda2
```

Swap-Space in Linux – Internal Structure

Linux swap internal structure

- Only anonymous memory is subject to swap (code pages can be re-read from files)
- Swap area is organized as an array of 4KB **page slots**
- **Swap map**: an array of integer counters for each page slot

```
Swap Area:

| Slot 0 | Slot 1 | Slot 2 | Slot 3 | Slot 4 |

Swap Map (counters):

| 1 | 0 | 3 | 0 | 1 |

used empty 3      empty used
           processes
           sharing
```

Counter Value	Meaning

Part 8

Storage Attachment

Storage Attachment – Overview

Methods for connecting storage to a system

Method	Description	Protocol
Host-Attached	Direct connection via SATA, SAS	AHCI, NVMe
NAS	File system access over network	NFS, CIFS/SMB
Cloud Storage	Internet-based storage service	REST API (S3, etc.)
SAN	Block-level access over dedicated network	Fibre Channel, iSCSI

```
Host-Attached:  Server ──[SATA/SAS/NVMe]──→ Disk
NAS:            Server ──[LAN (TCP/IP)]──→ NAS Device (file-level)
SAN:            Server ──[Dedicated Network]──→ Storage Array (block-level)
Cloud:          Server ──[Internet]──→ Cloud Provider (API-based)
```

NAS (Network-Attached Storage)

File-level storage access over a network

- **Protocols:** NFS (UNIX/Linux), CIFS/SMB (Windows)
- RPC (Remote Procedure Call) based communication over a general LAN
- File locking support → multiple clients can share files

iSCSI:

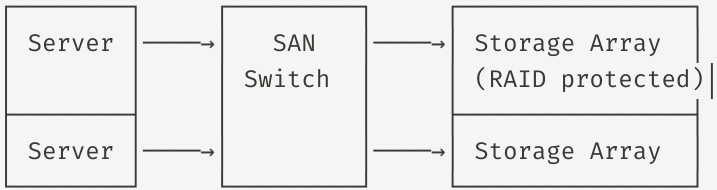
- Delivers SCSI protocol over IP networks
- **Block-level** access (NFS/CIFS is file-level)
- Host can use remote storage as if it were directly attached

NAS characteristics:

- Easy to set up, low cost
- Uses general network → storage I/O **consumes network bandwidth**
- Can become a performance bottleneck in large-scale environments

SAN (Storage-Area Network)

Connects servers and storage over a dedicated network



NAS vs SAN:

Item	NAS	SAN
Access level	File (file-level)	Block (block-level)
Network	General LAN	**Dedicated network**
Protocol	NFS, CIFS	FC, iSCSI

Cloud Storage

Remote storage service over the Internet

Property	NAS	Cloud Storage
Network	LAN	**Internet / WAN**
Access method	File system protocol	**API-based**
Failure handling	Hangs on disconnect	App pauses and resumes
Cost model	Equipment purchase	**Usage-based billing**

Representative services:

- Amazon S3 (Simple Storage Service)
- Microsoft OneDrive, Azure Blob Storage

Part 9

RAID Structure

RAID – Overview

RAID (Redundant Array of Independent Disks)

Combines multiple disks to improve **reliability and performance**

Purpose:

- **Redundancy:** enables data recovery in case of disk failure
- **Performance:** improves throughput via parallel I/O across multiple disks

Key techniques:

Technique	Description
Striping	Distributes data across multiple disks → **performance improvement**
Mirroring	Stores duplicate copies of data → **reliability improvement**

Reliability via Redundancy

MTBF (Mean Time Between Failures) Calculation

Single disk: MTBF = 100,000 hours **Array of 100 disks:** MTBF = 100,000 / 100 = **1,000 hours (about 42 days)**

The more disks there are, the higher the probability of failure somewhere → redundancy is needed

Mirrored Volume MTBF:

- Assuming two disks fail independently
- Individual MTBF = 100,000 hours, Mean Time to Repair = 10 hours
- MTBF (data loss) = 100,000^2 / (2 x 10) = **500,000,000 hours (about 57,000 years!)**

Caveats:

- In reality, the independent failure assumption does not always hold
- Power outages, natural disasters, manufacturing defects → simultaneous failures possible
- Disk aging → increased probability of second disk failure

Performance via Parallelism – Striping

Performance improvement through striping

Bit-level Striping:

- Distributes bits of each byte across multiple drives
- 8 drives: bit i is stored on drive i
- 8x faster data transfer rate, but all drives participate in every access

Block-level Striping (most common):

- With N drives, block i is stored on drive (i mod N)

```
4-Drive Block Striping:

Block 0 → Disk 0    Block 1 → Disk 1
Block 2 → Disk 2    Block 3 → Disk 3
Block 4 → Disk 0    Block 5 → Disk 1
Block 6 → Disk 2    Block 7 → Disk 3
 ...
```

Two goals of parallelism:

RAID Level 0

RAID 0 – Striping Only (No Redundancy)

```
Disk 0    Disk 1    Disk 2    Disk 3

 A1        A2        A3        A4
 A5        A6        A7        A8
 A9        A10       A11       A12
```

Item	Details
Minimum disks	2
Usable capacity	N x disk size (100%)
Read performance	N-fold improvement
Write performance	N-fold improvement

RAID Level 1

RAID 1 – Mirroring

```
Disk 0    Disk 1    Disk 2    Disk 3

 ┌─────┐   ┌─────┐   ┌─────┐   ┌─────┐
 │ A1  │   │ A1  │   │ A3  │   │ A3  │
 │ A2  │   │ A2  │   │ A4  │   │ A4  │
 │ ... │ | | ... | | | ... |   | ... |
 └─────┘   └─────┘   └─────┘   └─────┘

 mirror pair 1       mirror pair 2
```

Item	Details
Minimum disks	2
Usable capacity	N/2 x disk size (50%)
Read performance	Improved (can read from either side)

RAID Level 4

RAID 4 – Block Striping + Dedicated Parity Disk

```
Disk 0    Disk 1    Disk 2    Parity

 A1        A2        A3        Ap      Ap = A1 XOR A2 XOR A3
 B1        B2        B2        Bp
 C1        C2        C3        Cp
```

Item	Details
Minimum disks	3
Usable capacity	(N-1) x disk size
Fault tolerance	1 disk
Recovery method	XOR reverse computation from parity

RAID Level 5

RAID 5 – Block Striping + Distributed Parity

```
Disk 0    Disk 1    Disk 2    Disk 3    Disk 4

 A1        A2        A3        A4        Ap
 B1        B2        B2        Bp        B4
 C1        C2        Cp        C3        C4
 D1        Dp        D2        D3        D4
 Ep        E1        E2        E3        E4
```

Difference from RAID 4: **parity is distributed across all disks**

- Eliminates RAID 4's parity disk bottleneck problem
- Parity is not concentrated on a single disk

Item	Details
Minimum disks	3

RAID Level 6

RAID 6 – Dual Distributed Parity (P + Q)

```
Disk 0    Disk 1    Disk 2    Disk 3    Disk 4    Disk 5

 A1        A2        A3        A4        Ap        Aq
 B1        B2        B2        Bp        Bq        B4
 C1        C2        Cp        Cq        C3        C4
```

Item	Details
Minimum disks	4
Usable capacity	(N-2) x disk size
Fault tolerance	**2 disks**
Dual parity	P (XOR parity) + Q (Galois field math)

RAID 0+1 vs RAID 1+0 (RAID 10)

RAID 0+1: Stripe then Mirror

```
Stripe Set A                Stripe Set B (Mirror)

 ┌──────┬──────┐             ┌──────┬──────┐
 │ D0:A1│ D1:A2│             │ D2:A1│ D3:A2│
 │ D0:A3│ D1:A4│             │ D2:A3│ D3:A4│
 └──────┴──────┘             └──────┴──────┘

1 disk failure → entire stripe becomes unavailable
```

RAID 1+0 (RAID 10): Mirror then Stripe

```
Mirror Pair 0    Mirror Pair 1

 ┌──────┬──────┐   ┌──────┬──────┐
 │ D0:A1│ D1:A1│   │ D2:A2│ D3:A2│
 │ D0:A3│ D1:A3│   │ D2:A4│ D3:A4│
 └──────┴──────┘   └──────┴──────┘

1 disk failure → only that pair's mirror is used, rest remains normal
```

Why RAID 10 is preferable over RAID 0+1:

- Smaller blast radius on single disk failure

RAID Levels – Comprehensive Comparison

Level	Configuration	Reliability	Performance	Min Disks	Usable Capacity
RAID 0	Striping only	None	Highest	2	100%
RAID 1	Mirroring	1 disk fault tolerance	Read improved	2	50%
RAID 4	Striping + dedicated parity	1 disk fault tolerance	Parity bottleneck	3	(N-1)/N
RAID 5	Striping + distributed parity	1 disk fault tolerance	Improved over RAID 4	3	(N-1)/N
RAID 6	RAID 5 + dual parity	**2 disk fault tolerance**	Write overhead	4	(N-2)/N
RAID 10	Mirror + Stripe (1+0)	1 per pair	Excellent read/write	4	50%

RAID Level Selection Criteria:

RAID – Hot Spare & Implementation

Hot Spare:

- Does not store data, but is **automatically deployed as a replacement** when a failure occurs
- RAID 5 with 1 disk failure → hot spare immediately begins rebuild
- RAID auto-recovers without human intervention

RAID Implementation Layers:

Layer	Description
Software RAID	Implemented in OS kernel or volume manager (Linux md/mdadm)
HBA (Host Bus Adapter)	Hardware RAID controller card
Storage Array	Controller within an independent storage device (most common)

RAID Limitations and ZFS

What RAID does not protect against:

- File pointer errors, software bugs
- Incomplete write (torn write)
- RAID controller failure itself
- RAID only protects against **physical media errors**

ZFS's Innovative Approach:

- Stores checksums for all blocks (data + metadata)
- Checksums are stored in the **parent pointer**, not in the block itself

```
inode (contains checksum of data blocks)
  ├── checksum D1 → Data Block 1
  ├── checksum D2 → Data Block 2
  └── checksum D3 → Data Block 3
```

- Data corruption → checksum mismatch → automatically detected
- If a mirror exists, automatically recovers from the intact copy
- Integrates volume management + file system → storage pool concept

Lab

Disk Scheduling Algorithm Simulator

Lab – Disk Scheduling Simulator

Implement FCFS, SCAN, C-SCAN, LOOK, and C-LOOK algorithms to compare head movement distances

Input configuration:

- Request queue: list of cylinder numbers
- Current head position
- Disk size (maximum cylinder number)
- Movement direction (for SCAN, C-SCAN)

```python
def fcfs(requests, head):
    """FCFS: process requests in arrival order"""
    sequence = [head]
    total_movement = 0
    current = head

    for req in requests:
        total_movement += abs(current - req)
        sequence.append(req)
        current = req

    return sequence, total_movement
```

Lab – SCAN / C-SCAN Implementation

```python
def scan(requests, head, disk_size, direction="up"):
    """SCAN: move in one direction to the end, then reverse"""
    sequence = [head]
    total_movement = 0
    current = head

    left = sorted([r for r in requests if r < head])
    right = sorted([r for r in requests if r ≥ head])

    if direction == "up":
        run = right + [disk_size - 1] + left[::-1]
    else:
        run = left[::-1] + [0] + right

    for pos in run:
        total_movement += abs(current - pos)
        sequence.append(pos)
        current = pos

    return sequence, total_movement

def c_scan(requests, head, disk_size):
    """C-SCAN: service in one direction only, move from end to start"""
    sequence = [head]
```

Lab – LOOK / C-LOOK Implementation

```python
def look(requests, head, direction="up"):
    """LOOK: similar to SCAN but does not go to the end"""
    sequence = [head]
    total_movement = 0
    current = head

    left = sorted([r for r in requests if r < head])
    right = sorted([r for r in requests if r ≥ head])

    if direction == "up":
        run = right + left[::-1]
    else:
        run = left[::-1] + right

    for pos in run:
        total_movement += abs(current - pos)
        sequence.append(pos)
        current = pos

    return sequence, total_movement

def c_look(requests, head):
    """C-LOOK: similar to C-SCAN but does not go to the end"""
    sequence = [head]
```

Lab – Result Comparison and Visualization

```python
requests = [98, 183, 37, 122, 14, 124, 65, 67]
head = 53
disk_size = 200

algorithms = {
    "FCFS": fcfs(requests, head),
    "SCAN (down)": scan(requests, head, disk_size, "down"),
    "C-SCAN (up)": c_scan(requests, head, disk_size),
    "LOOK (down)": look(requests, head, "down"),
    "C-LOOK (up)": c_look(requests, head),
}

for name, (seq, dist) in algorithms.items():
    print(f"{name}: distance={dist}, path={seq}")
```

Expected results:

Algorithm	Total Movement
FCFS	640

Summary

Key concepts this week

Topic	Key Content
HDD Structure	Platter, Track, Sector, Cylinder, RPM
Access Time	Seek Time + Rotational Latency + Transfer Time
NVM (SSD)	NAND Flash, Wear Leveling, TRIM, Garbage Collection
NAND Flash	Page-level R/W, Block-level Erase, no overwrite
HDD Scheduling	FCFS, SCAN, C-SCAN, LOOK, C-LOOK
Linux I/O Scheduler	Deadline, NOOP, CFQ

Next Week

Week 14 – Security and Protection

- **Security:** CIA Triad, Program Threats, Network Threats, Cryptography
- **Protection:** Principle of Least Privilege, Access Matrix, RBAC
- **Authentication:** Password, Biometrics, Multifactor Authentication

Textbook: Ch 16, 17